

EE 105 Spring 2026

Homework 5 – upload to grade scope

(Due April 21, before class, late submission will incur 20% points/day penalty)

Instructions: Perform the following tasks based on the circuits and concepts discussed in class. Be sure to show all work where applicable.

1) Perceptron Model Calculation (Hand Calculation)

- Given a simple perceptron with weights $w_1 = 2$, $w_2 = -3$, $w_3 = 1.5$ and input values $x_1 = 1$, $x_2 = 0$, $x_3 = 1$, calculate the output assuming the threshold is 0. Use the perceptron rule:
 - Output** = 1 if the weighted sum is greater than the threshold, otherwise 0.

2) Activation Functions (Hand Calculation)

- For a neuron with input $x = 1.0$ and weight $w = 2.5$, calculate the output using:
 - Sigmoid Activation Function:**
 - ReLU Activation Function:**
 - Tanh Activation Function:**

3) Design a Neural Network Topology (Conceptual)

- Design a neural network to classify whether a person should attend a music festival. The inputs are:
 - Weather forecast (good or bad)
 - Availability of tickets (yes or no)
 - Friends attending (yes or no)
- Specify the number of inputs, weights, activation function, and output for this simple model.

4) Smallest Neural Network for MNIST Classification (Python Implementation)

- Your task is to design the smallest possible neural network with a single hidden layer that can classify the MNIST handwritten digits with at least **85% accuracy** for the MNIST TEST dataset.
- Calculate the number of floating point operations (meaning number of multiplications and additions) your neural networks requires to generate a single output. We have gone over this in class, so use the same approach we did there.
- **Requirements:**
 - Use python based on the code developed in the labs to implement your model.
 - The network should contain a **single hidden layer** and output predictions for **10 classes** (digits 0-9).
 - The network should be trained on the **MNIST** dataset.
 - Report the architecture you used, including the size of the hidden layer, what activation functions, and the achieved accuracy.

- 5) This question will be the conclusion of the ML lab session. In class you were shown how to train a neural network for hand written digit recognition on google colab. In this question we ask that you employ the neural network to classify camera images streamed from your arduino boards.

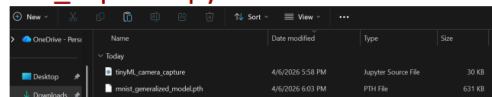
First you will need to upload the sketch to the arduino board to start streaming camera images.

- Change the jumper connection on the board
- Connect the board, change the selector to the camera module
- Open `tinyML_camera.ino`
 - Arduino/tinyML_camera
- Locate and add "Arduino_OV767X.zip"
 - Arduino
- Upload the sketch



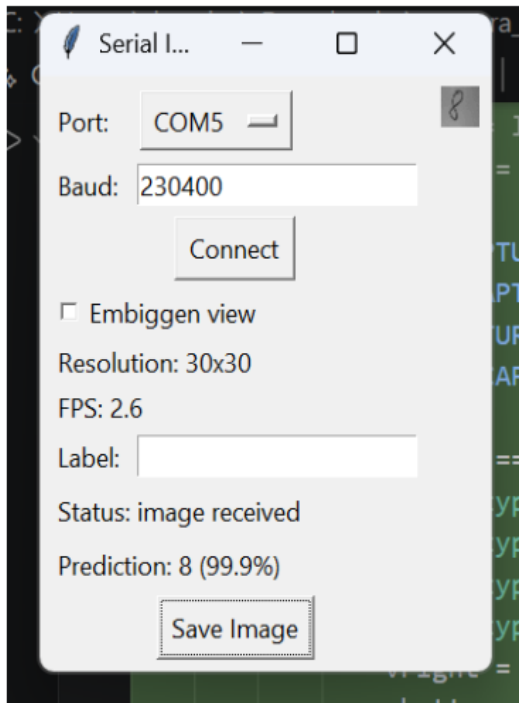
Second, you will need to use the downloaded neural network weights to classify the camera images streamed.

- Open `tinyML_camera_capture.ipynb`
 - Inside "Python"
- Move the downloaded checkpoint next to the notebook
- Select kernel same as last lab's
- Run all!
- Select the correct Port
- Capture



You should then write a custom digit on your notebook, point the camera to the digit, and capture the window. Your written digit should appear on the top right corner of the window. Further, the last line at the bottom of the window should display the Prediction and its corresponding confidence.

The captured image should look something like this:



Note that the handwritten digit is clearly visible on the top right corner, and the prediction is displayed at the bottom.

Report the handwritten digits, their predictions, and their corresponding confidences, for 3 different handwritten digits. For full credit, you will be required to submit the 3 final image captured on your PC.

